TF 15 - Parquet Skeleton External Format

Tracab Gateway

1.1

Table of Contents

Revisions **3**
Description **4**
Benefits 4
Format Specification **5**
Metadata Header 5
Skeleton Frame 6
Skeleton Target 7
Skeleton Target Part 7
Ball Target 8
Part Id mapping 8
TRACAB Skeletal Points **10**
TRACAB Coordinate System **11**
Examples **11**
Javascript 12
Python 13
Golang 14
Rust 15

Revisions

Rev.	Description	Date
1.0	Created initial version	2025-02-26
1.1	Renamed the document to describe the feed name	2025-09-19

Description

For downloading skeleton data from Tracab Gateway we use an Apache Parquet format.

The Parquet format is an open source, column-oriented data file format designed for efficient data storage and retrieval. The architecture of the format enables more efficient data compression and faster query performance.

The skeletal data of a match consists of (to date) 21points (parts) for each player, where each part is determined by an [X, Y, Z] set of coordinates, with a frequency of 25/30/50/60 times per second. The amount of data that it implies led to the use of this industry standard format for permanent storage.

Why not JSON? Although the live streamed skeleton data is in a readable JSON format, storing using the same format would take up an extensive amount of space and processing on our components.

Why not CSV, TSV? These types of files are hard to update once a solution has been implemented for the specific format.

Benefits

- **Language independency:** Official libraries exist in different languages (Some of them), implementations will not need to be tied to a specific one.
- **Size compression**: Data is stored in a binary format handled by the libraries that try to consume the least amount of space.
- **Flexibility**: Schema and data structure can be safely updated or extended, without interrupting client workflow because the schema is embedded in the same file as the data.
- **Industry standard:** it's a widely used format for big amounts of data that has been proven to work over the years.

- **Filtering and Searching:** Parquet has been created to be searchable as well so it makes it possible to look up specific frames if wanted, without having to search through the whole file.
- **Metadata:** Parquet files allow us to add some information about the file which we use for storing match data.

Format Specification

Below follows the specification of the format.

Metadata Header

Name	Type	Description
file_version	Integer	The version of the header
vendor_id	Integer	Game vendor id
game_id	Integer	Id of the Game
data_quality	Integer	Quality of the feed. 0 = raw data 1 = scrubbed data (post processed)
framerate	Integer	The hz frame rate of which the game is captured in
ai_clicker	Integer (0/1)	Indicating if an AI clicker was used for the game. 0 = no use of AI clicker 1 = use of AI clicker
phase_1_start	Integer	Start framecount for 1st half
phase_1_end	Integer	End framecount for 1st half
phase_2_start	Integer	Start framecount for 2nd half
phase_2_end	Integer	End framecount for 2nd half

phase_3_start	Integer	Start framecount for 1st extra time
phase_3_end	Integer	End framecount for 1st extra time
phase_4_start	Integer	Start framecount for 2nd extra time
phase_4_end	Integer	End framecount for 2nd extra time
phase_5_start	Integer	Start framecount for penalty time
phase_5_end	Integer	End framecount for penalty time
pitch_long	Integer	The long side of total tracking area (dm)
pitch_short	Integer	The short side of the total tracking area (dm)
pitch_padding_bottom	Integer	The bottom pitch padding between pitch and tracking area (dm)
pitch_padding_top	Integer	The top pitch padding between pitch and tracking area (dm)
pitch_padding_left	Integer	The left pitch padding between pitch and tracking area (dm)
pitch_padding_right	Integer	The right pitch padding between pitch and tracking area (dm)
home_team_id	Integer	Home team id
away_team_id	Integer	Away team id

Skeleton Frame

Name	Type	Description
version	Integer	Version of the skeleton frame format. Currently 1
frame_number	Integer	Framecount for current frame
type	Integer	Quality of the feed. 0 = raw data

		1 = filtered data (post processed)
skeleton_count	Integer	Number of skeleton targets
ball_exists	Integer 0/1	Indicating if ball exists in frame 0=Don't exists 1=Exists
skeletons	Array of skeletons Targets	Length is skeleton_count
ball	Ball target size	Not present if ball_exists is 0

Skeleton Target

Name	Type	Description
jersey_number	Integer	Jersey number of the player. Jersey -1 is unassigned. Head referee = 0, 1st Assistant = 1, 2nd Assistant = 2
team	Integer	Team identifier. 1=Home team 0=Away Team 3=Referee. Other values are used for internal purposes.
parts_count	Integer	Number of skeleton parts in target
parts	Array of Skeleton Target Part	Length is parts_count

Skeleton Target Part

Name	Type	Description
name	Integer	See Part id Mapping
position_x	Float	X position in cm from the center point

		of the field. According to the Tracab coordinate system.
position_y	Float	Y position in cm from the center point of the field. According to the Tracab coordinate system.
position_z	Float	Z position in cm from the center point of the field. According to the Tracab coordinate system.

Ball Target

Name	Type	Description
position_x	Float	X position in cm from the center point of the field. According to the Tracab coordinate system.
position_y	Float	Y position in cm from the center point of the field. According to the Tracab coordinate system.
position_z	Float	Z position in cm from the center point of the field. According to the Tracab coordinate system.
velocity_x	Float	X velocity in m/s
velocity_y	Float	Y velocity in m/s
velocity_z	Float	Z velocity in m/s

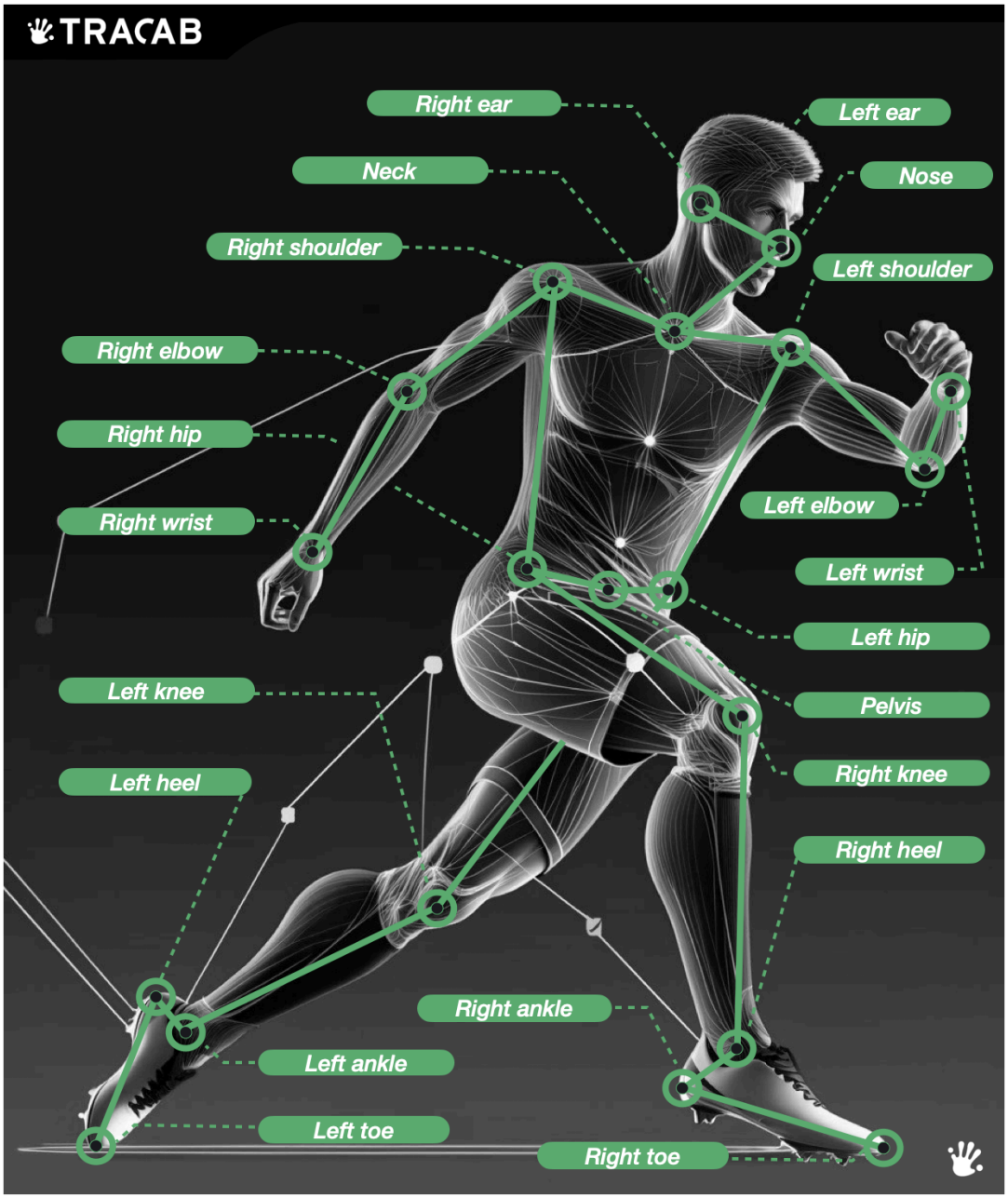
Part Id mapping

Any other number not specified in the following table should be treated as not assigned.

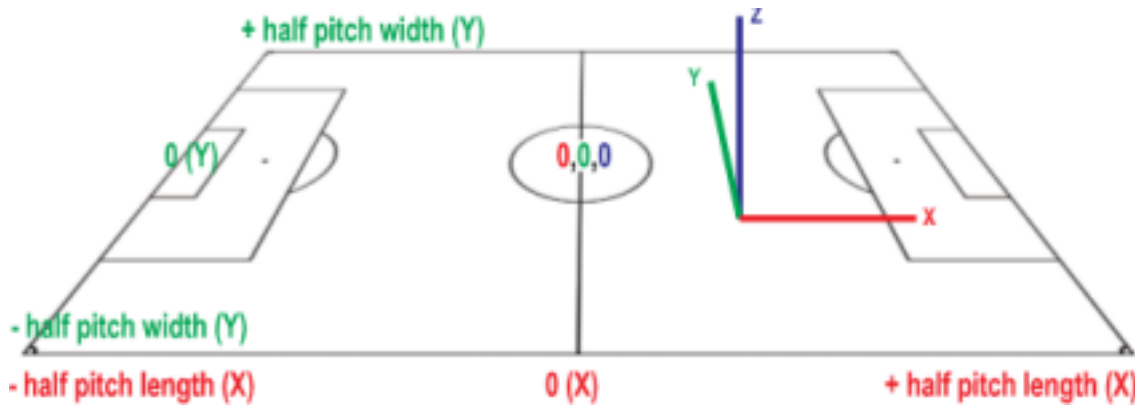
Name	Id
Left ear	1

Nose	2
Right ear	3
Left shoulder	4
Neck	5
Right shoulder	6
Left elbow	7
Right elbow	8
Left wrist	9
Right wrist	10
Left hip	11
Pelvis	12
Right hip	13
Left knee	14
Right knee	15
Left ankle	16
Right ankle	17
Left heel	18
Left toe	19
Right heel	20
Right toe	21

TRACAB Skeletal Points



TRACAB Coordinate System



Examples

Below follows some examples of how to read out the Parquet format from various programming languages.

The examples finds framecount: **3752711** in the file **999-202510-0.tsb.parquet**

Javascript

read_parquet.js

```javascript
import duckdb from "duckdb";

(async () => {
  const db = new duckdb.Database(":memory:");
  db.all(
      "SELECT * FROM READ_PARQUET('./999-202510-0.tsb.parquet') WHERE
frame_number=3752711 LIMIT 1",
      function (err, res) {
            if (err) {
                  throw err;
            }
            console.log(res);
      },
  );
})();
```


TRACAB

Python

read_parquet.py

```python
import pyarrow.parquet as pq

if __name__ == "__main__":
	parquet_file = pq.ParquetDataset(
		"999-202510-0.tsb.parquet",
		filters=[("frame_number", "=", 3752711)]
	)
	print(parquet_file.read().to_pandas())
```

Golang

Java

```go
package main

import (
	"log"
	"os"

	"github.com/parquet-go/parquet-go"
)

func main() {
	file, _ := os.Open("./999-202510-0.tsb.parquet")
	s, _ := file.Stat()
	fr, _ := parquet.OpenFile(file, s.Size())
	log.Printf("Num rows: %d", fr.NumRows())
	type RowType struct {
		Frame_number     int64    `parquet:"frame_number"`
		Version          int      `parquet:"version"`
		SkeletonCount    int      `parquet:"skeleton_count"`
		FrameNumber      int      `parquet:"frame_number"`
		UtcTime          float32  `parquet:"utc_time"`
		TrackedBallExists bool    `parquet:"ball_exists"`
		Type             int      `parquet:"type"`
	}
	schema := parquet.SchemaOf(new(RowType))
	reader := parquet.NewReader(file, schema)

	var row RowType
	for {
		err := reader.Read(&row)
		if err != nil {
			log.Fatalf("Error reading file: %v", err)
		}
		if row.Frame_number == 3752711 {
			log.Printf("%v", row)
			break
		}
	}
}
```

Rust

```
None
use parquet::basic::{Repetition, Type as PhysicalType};
use parquet::file::reader::{FileReader, SerializedFileReader};
use parquet::schema::types::{SchemaDescriptor, Type};
use std::fs::File;
use std::sync::Arc;

fn main() {
        let file = File::open("./999-202510-0.tsb.parquet").unwrap();
        let serialized_reader = SerializedFileReader::new(file).unwrap();
        let field_a = Type::primitive_type_builder("frame_number", PhysicalType::INT64)
                .with_repetition(Repetition::REQUIRED)
                .build()
                .unwrap();

        let field_b = Type::primitive_type_builder("skeleton_count", PhysicalType::INT64)
                .with_repetition(Repetition::REQUIRED)
                .build()
                .unwrap();

        let schema = Type::group_type_builder("Frame")
                .with_fields(vec![Arc::new(field_a), Arc::new(field_b)])
                .build()
                .unwrap();

        let projection = SchemaDescriptor::new(Arc::new(schema.clone()));

        let mut rows = serialized_reader
                .get_row_iter(Some(projection.root_schema().clone()))
                .unwrap();

        while let Some(record) = rows.next() {
                let rec = record.unwrap();
                let f_i = rec.get_column_iter().nth(0).unwrap().1;
                if f_i.eq(&parquet::record::Field::Long(3752711)) {
                        println!("{:?}", rec);
                break;
                }
        }
}
```